CIS5200: Machine Learning Spring 2026

5: Gradient Descent

Date: January 29, 2026

So far in this course, we've looked at a couple of machine learning algorithms: the Perceptron and k nearest neighbors. While these algorithms (in particular k-NN) are great tools, they don't really offer us a more general framework in which we have a lot of freedom to do learning. Instead, they had specific assumptions and insights that led to single algorithms.

That's all about to change. Over the next couple of lectures, we'll introduce the basis for almost all of machine learning: *optimization*. In particular, we will look at perhaps the most intuitive and powerful tool for training models: *gradient descent*.

Note: While we'll study gradient descent in its purest form, this algorithm forms the backbone of modern deep learning. The same principles we'll discover here power the training of massive language models like GPT-3 (175B parameters) and vision models like DALL-E, albeit with sophisticated modifications for scale and efficiency. Understanding the fundamentals we cover today will help you grasp advanced concepts in deep learning, from stochastic and mini-batch variants to adaptive learning rate methods like Adam and AdaGrad, and even second-order methods and their approximations.

A reminder: the training error. A few lectures ago, we introduced the concept of measuring the error a model $h(\mathbf{x})$ by averaging some *loss function* over the predictions of the model and the true labels:

$$\hat{R}(h) = \frac{1}{n}\sum_{i=1}^{n} \ell(h(\mathbf{x}_i), y_i).$$

While we've discussed how we can use this idea to *measure* the error, it's not really clear how algorithms like the perceptron or k-NN attempt to directly achieve low error. A really natural idea – and indeed the foundation of optimization based machine learning – is to **directly find the** h **that minimizes the empirical risk** $\hat{R}(h)$:

$$h^* = \arg\min_{h\in\mathcal{H}} \hat{R}(h)$$

As written above, this seems essentially intractable: we're being asked to find the best model h, perhaps in some infinitely large family of models $\mathcal{H}$, that minimizes the empirical risk. This minimization becomes less abstract once we fix a particular family of models $\mathcal{H}$, however. As a particular example that we'll study a lot, recall that the perceptron learned a **linear model** of the form $h(x) = w^\top x + b$. In some sense, functions of the form $w^\top x + b$ *parameterize* the entire set $\mathcal{H}$ of linear models. Therefore, we can do optimization over w and b directly. Now, instead of writing the optimization problem above as a minimization problem over the entire function h, we

can directly write it as a minimization problem over w and b:

$$\begin{aligned} h^*(x) &= {w^*}^\top x + b^* \\ w^*, b^* &= \arg\min_{w,b} \hat{R}(h) \\ &= \arg\min_{w,b} \frac{1}{n} \sum_{i=1}^{n} \ell(w^\top x + b, y_i) \end{aligned} \tag{1}$$

It turns out that Equation 1 now looks like an optimization problem that we *will* know how to solve directly, at least for a pretty broad range of loss functions ℓ. We just need to introduce the machinery to do so.

Abstracting away from linear models (and indeed machine learning) for a moment, we essentially want to study problems of the form:

$$\text{minimize over } w \qquad \underbrace{F(w)}_{\text{objective}}$$

where $F : \mathbb{R}^d \to \mathbb{R}$ is our objective function. While optimization problems can have constraints ($w \in \mathcal{C}$ for some set $\mathcal{C} \subseteq \mathbb{R}^d$), in this lecture we'll focus on the unconstrained case where w can take any value in $\mathbb{R}^d$. This is sufficient for many machine learning problems, including linear regression and logistic regression. We'll study constrained optimization in a future lecture.

1 Moving downhill

The trick to minimizing a function $F(w)$ when you don't know much about it is to assume it's much simpler than it is. If $F(w)$ were just a line, the slope of the line would tell us which way to go if we want to decrease $F(w)$. If $F(w)$ were a quadratic function, we'd know how to minimize it immediately. The basic idea in gradient based optimization is to approximate $F(w)$ locally at some location w with one of these much simpler functions, solve the easy problem, and then repeat.

How do we approximate $F(w)$ with a linear or quadratic function? Well, it's finally time to put those Taylor approximations to good use! The first and second order Taylor expansion of $F(w)$ at some location w are given by:

$$F(w+v) \approx F(w) + \nabla F(w)^\top v \qquad \text{(first-order)}$$

$$F(w+v) \approx F(w) + \nabla F(w)^\top v + \frac{1}{2} v^\top H v \qquad \text{(second-order)}$$

Here, $\nabla F(w)$ is the gradient and H is the Hessian, e.g. the matrix so that $H_{ij} = \frac{\partial^2 F}{\partial w_i \partial w_j}$.

1.1 Gradient Descent

The Taylor expansion gives us a nice approximation to $F(w)$ that lets us reason locally about how to decrease the objective. How do we use these approximations? The fundamental idea is, given some current w_t, we will use this approximation to choose a reasonable v so that $F(w+v) < F(w)$, and therefore we can update $w_{t+1} = w_t - \eta_t v$ and get a better model h.

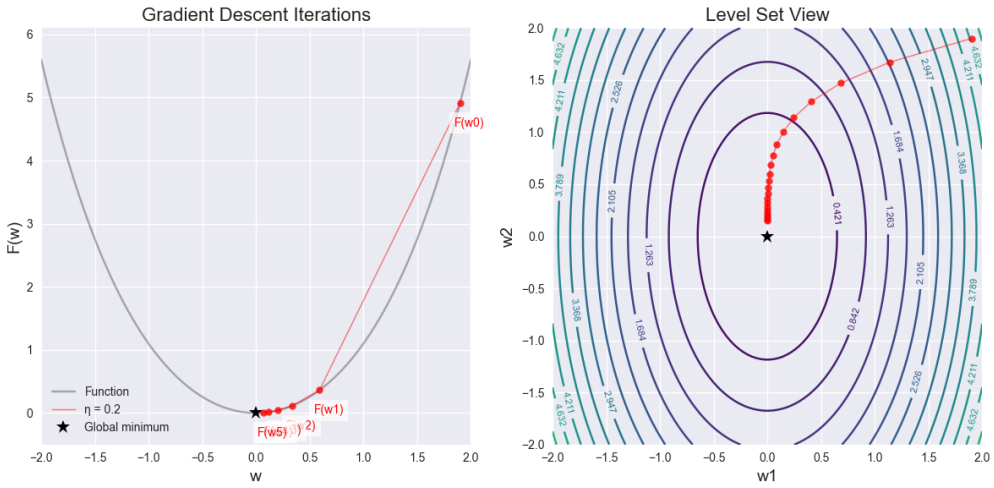

Figure 1: Two views of gradient descent optimization. Left: A one-dimensional quadratic function showing the trajectory of gradient descent with learning rate $\eta = 0.2$. The red dots mark each iteration, starting from w_0 and converging to the global minimum (black star). Right: Level set view of a two-dimensional quadratic function, where each contour represents points of equal function value. The red dots show the trajectory of gradient descent with the same fixed learning rate. At each point, the update direction is perpendicular to the level sets, with denser contours indicating regions of larger gradients (steeper slopes).

How do we find the best v? Using the first-order Taylor approximation around w_t,

$$F(w_t + \Delta) \approx F(w_t) + \nabla F(w_t)^\top \Delta.$$

To decrease F, we want the change $\nabla F(w_t)^\top \Delta$ to be negative. For fixed size $\|\Delta\|_2 = \eta_t$, then by Cauchy–Schwarz,

$$\nabla F(w_t)^\top \Delta \geq -\|\nabla F(w_t)\|_2 \cdot \|\Delta\|_2 = -\eta_t \|\nabla F(w_t)\|_2,$$

with equality when Δ points in the direction of $-\nabla F(w_t)$. Plugging this into the first-order Taylor approximation gives

$$F(w_{t+1}) \approx F(w_t) - \eta_t \|\nabla F(w_t)\|_2^2,$$

so (for small enough η_t) we should decrease the objective each step. This is progress!

This gives us the famous Gradient Descent algorithm:

Algorithm 1: Gradient Descent (GD)

Initialize $w_1 \in \mathbb{R}^d$
while $t = 1, 2, \ldots, T$ **do**
    Update $w_{t+1} = w_t - \eta_t \nabla F(w_t)$
end

At each iteration, the algorithm computes the gradient of $F(\cdot)$ at the current location w_t, and then simply moves a little bit in the direction of the negative gradient. Here η_t are known as the *learning rates*. Usually we use a stopping condition to terminate the algorithm, for instance, when $F(w_t) \leq \epsilon$ or when $\|\nabla F(w_t)\|_2 \leq \epsilon$.

How to choose η_t Choosing a good learning rate is paramount to the success of gradient descent. If the learning rate is too large, then we could diverge, that is, instead of making progress at each time step, we could instead make negative progress. If the learning rate is too small, then it could take us a very long time to converge to a good solution. One way of setting the learning rate is "guess and check": find a learning rate where things start to diverge, and make the learning rate smaller than that.

In modern machine learning, several very crazy learning rate schedules work well, counter-intuitive to what we can get guarantees for in convex problems. This is a very active area of research.

It's important to wonder at this point what conditions might be useful for gradient descent working *quickly* or even at all, and we will cover an introduction to this.

1.2 Newton's method

In Newton's method, we make use of the second order Taylor expansion of the function,

$$F(w+v) \approx F(w) + \nabla F(w)^\top v + \frac{1}{2} v^\top H v$$

Like in gradient descent, the fundamental question is how to use this approximation to choose a reasonable v. Unlike in gradient descent where we approximated the function with a line that gives us a direction to follow, here we get a quadratic function that we can minimize directly and *find the best v under this approximation.* To do this, we can take the derivative of the approximation with respect to v, set it equal to zero and solve:

$$\nabla_v \left[F(w) + \nabla F(w)^\top v + \frac{1}{2} v^\top H v \right] = \nabla F(w) + Hv$$
$$\nabla F(w) + Hv = 0 \Rightarrow v = -H^{-1} \nabla F(w)$$

In settings where the quadratic approximation is accurate (this is the same question we had for standard gradient descent!), this update rule ($w_{t+1} \leftarrow w_t - H^{-1} \nabla F(w)$) converges *extremely* fast. Interestingly, note the lack of step size in standard Newton's method: this is a common source of divergence. If there are directions of the function that are extremely shallow, the inverse Hessian will have extremely large eigenvalues, leading to very large steps in those directions.

2 When is gradient descent good?

In principle, the recipe for using gradient descent in machine learning is pretty simple: specify a bunch of parameters that define a hypothesis class $\mathcal{H}$, pick a loss function that specifies how bad a particular $h \in \mathcal{H}$ is on your training data, then use gradient descent to make the loss go down! The problem with this is that not all losses are "nice" for gradient descent. For example, consider the 0/1 loss over a decision rule of the form $h(x) = \text{sgn}(w^\top x)$:

$$\ell_{0/1}(\text{sgn}(w^\top x), y) = \begin{cases} 0 & \text{if } \text{sgn}(w^\top x) = y \\ 1 & \text{otherwise.} \end{cases}$$

As an exercise, I'd encourage you to think about the derivative of this loss function with respect to w. Gradient descent wouldn't achieve very much here!

In the rest of these notes, we're going to a go through lot of theory that describes settings in which we know for a fact that gradient descent works, or sometimes even works extremely well. In particular, there are some properties that $F(w)$ can have that make gradient descent "nice". It's important to emphasize that these properties are not necessarily *required* for gradient descent to function – some models like neural networks recklessly abandon many of these nice properties. Nevertheless, for a long time it was thought in machine learning that these properties were very important to have, and many classic machine learning algorithms that we'll learn satisfy them. In particular, there are two properties we will study that make gradient descent extra nice: **smoothness** and **(strong) convexity.**

3 Convex functions and sets

The first interesting properties that our function $F(w)$ and constraint set $\mathcal{C}$ can satisfy is **convexity**. As we'll see, one of the key things that convexity will guarantee is that, if gradient descent finds a stationary point (e.g., a point w so that $\nabla F(w) = 0$), that stationary point is guaranteed to be a global optimum! In addition, a function can be **strongly** convex, which (along with smoothness) will result in very fast convergence.

Definition 1 (Convex function)**.** *A function $F : \mathbb{R}^d \to \mathbb{R}$ is convex if for all $w, w' \in \mathbb{R}^d$ and $\alpha \in [0, 1]$,*

$$F\left(\alpha w + (1-\alpha)w'\right) \leq \alpha F(w) + (1-\alpha)F(w').$$

Definition 2 (Convex set)**.** *A set $\mathcal{C} \subseteq \mathbb{R}^d$ is convex if for all $w, w' \in \mathcal{C}$ and $\alpha \in [0, 1]$,*

$$\alpha w + (1-\alpha)w' \in \mathcal{C}.$$

3.1 First and second order characterizations of convex functions

Suppose convex function F is twice differentiable, then the following statements are true:

- for all $w, w' \in \mathbb{R}^d$, $F(w') \geq F(w) + \nabla F(w)^\top (w' - w)$.
- for all $w \in \mathbb{R}^d$, $\nabla^2 F(w) \succeq 0$, that is, $\nabla^2 F(w)$ is a PSD matrix.

The first statement says that the function always lies above the tangent at any point. The second says that the curvature of the function is always non-negative, that is, never downwards.

Try to show that these statements are true from the definition of convex functions. For the latter one, try to show in 1-dimension first.

Theorem 3. *For any convex differentiable function F, any w that satisfies $\nabla F(w) = 0$ is a global minimum of F.*

Proof. From the first property above, we have for all w',

$$F(w') \geq F(w) + \nabla F(w)^\top (w' - w) \implies F(w') \geq F(w),$$

since $\nabla F(w) = 0$. □

This property highlights the local to global phenomenon, if the minimum is a local minimum then it is a global minimum. Also, this minimum is unique if F is strictly convex.

4 GD for Smooth Functions

Let us look at convex functions that are smooth, that is, they don't change value very quickly. More formally,

Definition 4 (Smooth function)**.** *A function $F : \mathbb{R}^d \to \mathbb{R}$ is L-smooth if for all $w, w' \in \mathbb{R}^d$,*

$$F(w') \leq F(w) + \nabla F(w)^\top (w' - w) + \frac{L}{2}\|w' - w\|_2^2.$$

One intuitive reason that this is called "smoothness" is that this condition is equivalent to saying that the ∇F is L-Lipschitz, that is, for all $w, w' \in \mathbb{R}^d$

$$\|\nabla F(w) - \nabla F(w')\|_2 \leq L\|w - w'\|_2.$$

This intuitively means that the gradient can only change so quickly as you move away from w.

For our purposes, the key interpretation of what L-smoothness means intuitively is that there is a quadratic function that takes the value $F(w)$ at w and upper bounds the function otherwise. This property is extremely useful, and–as we will see–directly implies that gradient descent works! One way to see that this is useful is to play the same trick that we did in Newton's method and minimize the quadratic expansion implied by smoothness:

$$\min_{w'} F(w) + \nabla F(w)^\top (w' - w) + \frac{L}{2}\|w' - w\|_2^2.$$

Taking the derivative of the above expression with respect to w' and setting it equal to zero, we find

$$w' = w - \frac{1}{L}\nabla F(w).$$

which is exactly the gradient step with learning rate $\eta = 1/L$. Here's the trick though: in Newton's method, all we knew was that the second order Taylor expansion was in some sense "approximating" the function around w. The quality of that approximation could be poor, causing Newton's approximation to fail.

4.1 Observation 1: Gradient descent is nonincreasing on smooth functions.

Unlike in Newton's method, if we know the function is L smooth, the expression above is not just an approximation: **it's an upper bound!**. Rearranging the above minimizer, $w' = w - \frac{1}{L}\nabla F(w)$ we find that $\nabla F(w) = L(w - w')$. Observe that $w - w' = -(w' - w)$. This is useful because plugging

this into the upper bound we see:

$$\begin{aligned}
F(w') &\leq F(w) + \nabla F(w)^\top (w' - w) + \frac{L}{2}\|w' - w\|_2^2 \\
&= F(w) + L(w - w')^\top (w' - w) + \frac{L}{2}\|w' - w\|_2^2 \\
&= F(w) - L(w' - w)^\top (w' - w) + \frac{L}{2}\|w' - w\|_2^2 \\
&= F(w) - L\|w' - w\|_2^2 + \frac{L}{2}\|w' - w\|_2^2 \\
&= F(w) - \frac{L}{2}\|w' - w\|_2^2
\end{aligned}$$

This is very important – it means that after updating $w' = w - \frac{1}{L}\nabla F(w)$, $F(w')$ is less than or equal to $F(w)$ by a known amount, namely $\frac{L}{2}\|w' - w\|_2^2$. In fact, let's formalize this as a Lemma:

Lemma 5. *Let $F(w)$ be an L-smooth convex function. In iteration t of gradient descent, when we update $w_{t+1} = w_t - \frac{1}{L}\nabla F(w_t)$, we have that*

$$\underbrace{F(w_{t+1})}_{\text{new value}} \leq \underbrace{F(w_t)}_{\text{old value}} - \underbrace{\frac{L}{2}\|w_{t+1} - w_t\|_2^2}_{\geq 0},$$

or equivalently,

$$F(w_{t+1}) - F(w_t) \leq -\frac{L}{2}\|w_{t+1} - w_t\|_2^2, \tag{2}$$

In other words, gradient descent is nonincreasing in $F(w)$ in each iteration.

4.2 Observation 2: Convergence on smooth convex functions (statement).

The fact that gradient descent produces non-increasing function values on smooth functions is interesting: it means that it'll never make anything **worse**. The interesting question, of course, is whether we actually get closer to the global optimum. The answer is yes, and we can quantify the rate.

Lemma 6. *Let F be an L-smooth convex function. In iteration t of gradient descent, when we update $w_{t+1} = w_t - \frac{1}{L}\nabla F(w_t)$, we have*

$$F(w_{t+1}) - F(w_*) \leq \frac{L}{2}\left(\|w_t - w_*\|_2^2 - \|w_{t+1} - w_*\|_2^2\right). \tag{3}$$

The proof of Lemma 6 is in the appendix.

4.3 Convergence Rate for Smooth Convex Functions

Using Lemma 6 and Lemma 5, we can prove the convergence of gradient descent.

Theorem 7. *Suppose we run GD on L-smooth function F with fixed constant learning rate $\eta_t = 1/L$. Then after T iterations, we have*

$$F(w_{T+1}) - F(w_*) \leq \frac{L\|w_1 - w_*\|_2^2}{2T}$$

where w_* is the global minimum.

Let us interpret this result. Suppose we initialize at $w_1 = 0$ and $\|w_*\|_2 = 1$, then this implies that after T iterations

$$F(w_{T+1}) - F(w_*) \leq \frac{L}{2T}.$$

This implies that after $T = \frac{L}{2\epsilon} = O(1/\epsilon)$ steps, we get that $F(w_{T+1}) - F(w_*) \leq \epsilon$. Therefore we say that gradient descent has a convergence rate of $O(1/T)$.

Proof. Summing Equation 3 over $t = 1, \ldots, T$ telescopes. Concretely,

$$\sum_{t=1}^{T} \left(\|w_t - w_*\|_2^2 - \|w_{t+1} - w_*\|_2^2\right) = \underbrace{\left(\|w_1 - w_*\|_2^2 - \|w_2 - w_*\|_2^2\right)}_{\text{Term for } t=1} + \underbrace{\left(\|w_2 - w_*\|_2^2 - \|w_3 - w_*\|_2^2\right)}_{\text{Term for } t=2} + \cdots$$

Notice that the $-\|w_2 - w_*\|_2^2$ in the term for $t = 1$ cancels with the $\|w_2 - w_*\|_2^2$ in the term for $t = 2$, and so on. The only two terms that do not cancel are $\|w_1 - w_*\|_2^2$ from the first term and $-\|w_{T+1} - w_*\|_2^2$ from the last term. Therefore,

$$\sum_{t=1}^{T} \left(F(w_{t+1}) - F(w_*)\right) \leq \frac{L}{2}\left(\|w_1 - w_*\|_2^2 - \|w_{T+1} - w_*\|_2^2\right) \leq \frac{L}{2}\|w_1 - w_*\|_2^2.$$

By Lemma 5, $F(w_{T+1}) \leq F(w_{t+1})$ for all $t \leq T$, so

$$T \cdot \left(F(w_{T+1}) - F(w_*)\right) \leq \sum_{t=1}^{T} \left(F(w_{t+1}) - F(w_*)\right) \leq \frac{L}{2}\|w_1 - w_*\|_2^2.$$

Dividing by T gives the standard $O(1/T)$ convergence rate. □

5 GD on Smooth and Strongly Convex Functions

Let us look at convex functions that are strongly convex, that is, they have high curvature at all points. More formally,

Definition 8 (Strongly Convex function)**.** *A function $F : \mathbb{R}^d \to \mathbb{R}$ is μ-strongly convex if for all $w, w' \in \mathbb{R}^d$,*

$$F(w') \geq F(w) + \nabla F(w)^\top (w' - w) + \frac{\mu}{2}\|w' - w\|_2^2.$$

For strongly convex functions, we can get a faster rate of convergence.

Theorem 9. *Suppose we run GD on L-smooth and μ-strongly convex function F with fixed constant learning rate $\eta_t = 1/L$ for all $t \in [T]$. Then for all time τ, we have*

$$\|w_{\tau+1} - w_*\|_2^2 \leq \left(1 - \frac{\mu}{L}\right)^{\tau} \|w_1 - w_*\|_2^2.$$

where w_* is the global minimum.

Let us interpret this result. Suppose we initialize at $w_1 = 0$ and $\|w_*\|_2 = 1$, then this implies that after T iterations

$$\|w_{T+1} - w_*\|_2^2 \leq \left(1 - \frac{\mu}{L}\right)^{T}.$$

This implies that after $T = \frac{L}{\mu}\log(1/\epsilon) = O(\log(1/\epsilon))$ steps, we get that $\|w_{T+1} - w_*\|_2 \leq \epsilon$. Therefore we say that gradient descent has a convergence rate of $O(\exp(-T))$.

Remark. The key parameter is the condition number $\frac{L}{\mu}$. When μ is close to L (well-conditioned), we converge quickly; when $\mu \ll L$ (ill-conditioned), we converge more slowly.

A Appendix: Proofs and additional material

A.1 Proof of Lemma 6

We prove the per-iteration inequality Equation 3.

First, observe two key properties:

1. By convexity of $F(w)$, $F(w_*) \geq F(w_t) + \nabla F(w_t)^\top (w_* - w_t)$. This follows from the first property in Section 3.1.
2. As we observed above, the update rule to get w_{t+1} implies that $\nabla F(w_t) = L(w_t - w_{t+1})$.

Plugging the expression for the gradient from the the second bullet into the convexity property in the first bullet:

$$\begin{aligned} & F(w_*) \geq F(w_t) + L(w_t - w_{t+1})^\top (w_* - w_t) \\ \implies & F(w_t) - F(w_*) \leq -L(w_t - w_{t+1})^\top (w_* - w_t) \\ \implies & F(w_t) - F(w_*) \leq L(w_t - w_{t+1})^\top (w_t - w_*) \end{aligned} \tag{4}$$

Now for some algebraic trickery. By subtracting and adding w_t:

$$\begin{aligned} \|w_{t+1} - w_*\|_2^2 &= \|w_{t+1} - w_t + w_t - w_*\|_2^2 \\ &= \|w_{t+1} - w_t\|_2^2 + \|w_t - w_*\|_2^2 - 2(w_t - w_{t+1})^\top (w_t - w_*). \end{aligned}$$

Rearranging:

$$(w_t - w_{t+1})^\top (w_t - w_*) = \frac{1}{2}\left(\|w_{t+1} - w_t\|_2^2 + \|w_t - w_*\|_2^2 - \|w_{t+1} - w_*\|_2^2\right). \tag{5}$$

Alternative Derivation via Law of Cosines: Equation (5) can also be derived using the law of cosines! Consider the triangle formed by points w_t, w_{t+1}, and w_*. The law of cosines states that for any triangle with sides a, b, and c, and angle θ opposite to side c:

$$c^2 = a^2 + b^2 - 2ab\cos(\theta)$$

In our case, mapping the sides to match the law of cosines equation:

- Side $c = \|w_{t+1} - w_*\|_2$ (opposite to angle θ)
- Side $a = \|w_t - w_*\|_2$ (adjacent to angle θ)
- Side $b = \|w_{t+1} - w_t\|_2$ (adjacent to angle θ)
- The angle θ is between sides a and b, with $\cos(\theta) = \frac{(w_t - w_{t+1})^\top (w_t - w_*)}{\|w_{t+1} - w_t\|_2 \|w_t - w_*\|_2}$

Plugging these into the law of cosines:

$$\|w_{t+1} - w_*\|_2^2 = \|w_t - w_*\|_2^2 + \|w_{t+1} - w_t\|_2^2 - 2\|w_t - w_*\|_2 \|w_{t+1} - w_t\|_2 \cos(\theta)$$

Substituting the expression for $\cos(\theta)$ and rearranging gives us exactly equation (5).

Plugging this into Equation 4, we see that

$$F(w_t) - F(w_*) \leq \frac{L}{2}\left(\|w_{t+1} - w_t\|_2^2 + \|w_t - w_*\|_2^2 - \|w_{t+1} - w_*\|_2^2\right)$$

Now, if we add Equation 2 from Lemma 5 to the equation above:

$$\begin{aligned} & F(w_{t+1}) - F(w^*) \leq -\frac{L}{2}\|w_{t+1} - w_t\|_2^2 + \frac{L}{2}\left(\|w_{t+1} - w_t\|_2^2 + \|w_t - w_*\|_2^2 - \|w_{t+1} - w_*\|_2^2\right) \\ \implies & F(w_{t+1}) - F(w^*) \leq \frac{L}{2}\left(\|w_t - w_*\|_2^2 - \|w_{t+1} - w_*\|_2^2\right) \end{aligned} \tag{6}$$

Since $F(w^*) \leq F(w_{t+1})$ because w_* is the minimizer, we know that $0 \leq F(w_{t+1}) - F(w^*)$. Therefore

$$\begin{aligned} & 0 \leq F(w_{t+1}) - F(w^*) \leq \frac{L}{2}\left(\|w_t - w_*\|_2^2 - \|w_{t+1} - w_*\|_2^2\right) \\ \implies & 0 \leq \frac{L}{2}\left(\|w_t - w_*\|_2^2 - \|w_{t+1} - w_*\|_2^2\right) \\ \implies & \|w_{t+1} - w_*\|_2^2 \leq \|w_t - w_*\|_2^2 \end{aligned}$$

This means that in each iteration, w_{t+1} gets closer to w^*!